
Práctica 9: Pasadas de geometría

Objetivo: Creación de pasadas con geometría en URP

1. Instrucciones

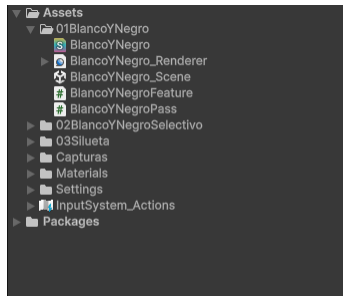
La práctica se realiza sobre un proyecto de Unity en el que se van añadiendo escenas que utilizan renderers distintos configurados para utilizar las *Render features* implementadas en cada apartado.

Se entregará el proyecto completo de Unity, dejando únicamente los directorios necesarios (Assets, Packages y ProjectSettings). En el directorio Assets habrá, además, una carpeta Capturas con imágenes de pantalla.

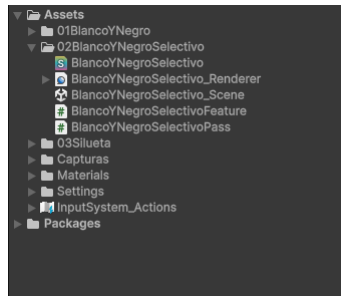
Por cada apartado de la práctica aparecerá un directorio con todos los ficheros creados en ese apartado. El nombre del directorio comenzará con el número del apartado seguido de su título en *camel case*. Por ejemplo, para el primer apartado habrá un directorio llamado 01BlancoYNegro. Los nombres de los ficheros del directorio comenzarán siempre con el título en *camel case* (en el primer apartado, todos empezarán por BlancoYNegro). Deberán aparecer:

- Un fichero con la escena. Su nombre terminará con el sufijo `_Scene`.
- Un fichero con el *renderer* utilizado por la cámara. Su nombre terminará con el sufijo `_Renderer`.
- Un fichero con el shader implementado para conseguir el efecto.
- Dos ficheros con la *render feature* y *render pass* implementadas en el apartado. Sus nombres terminarán con los sufijos `Feature` y `Pass` y coincidirán con los nombres de las clases que implementan.

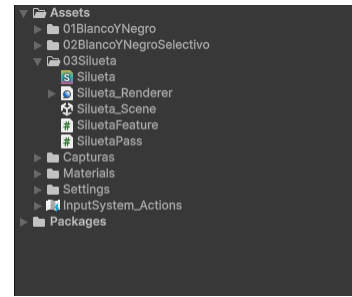
Por cada apartado, además, deben entregarse tres capturas en el directorio Capturas. El nombre de las tres capturas comenzarán con el número de apartado seguido del título (igual que el nombre del directorio) y terminarán con los sufijos `_Escena` (para la captura con la escena del juego), `_FrameDebugger` (para la captura con el frame FrameDebugger en donde se vea la etapa añadida) y `_RenderGraph` (para la captura en la que se vea el render graph).



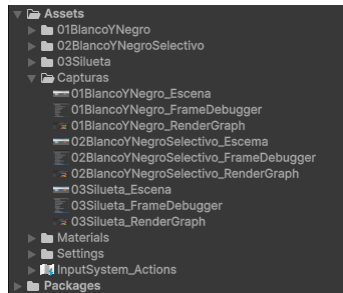
(a) Contenido del directorio del primer apartado



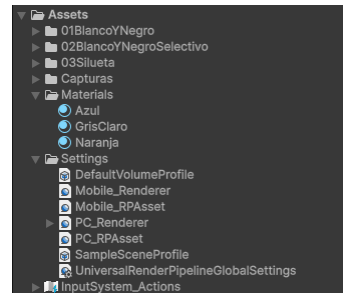
(b) Contenido del directorio del segundo apartado



(c) Contenido del directorio del tercer apartado



(d) Contenido del directorio Capturas



(e) Contenido del directorio Materials y Settings

Figura 1: Contenido del directorio Assets de la entrega

El proyecto hará uso del *render pipeline asset* por defecto creado por Unity al construir el proyecto y se configurará para que tenga disponibles los renderers de cada apartado.

2. Punto de partida

La práctica parte de un proyecto 3D en Unity que utilice el *Universal Render Pipeline*.

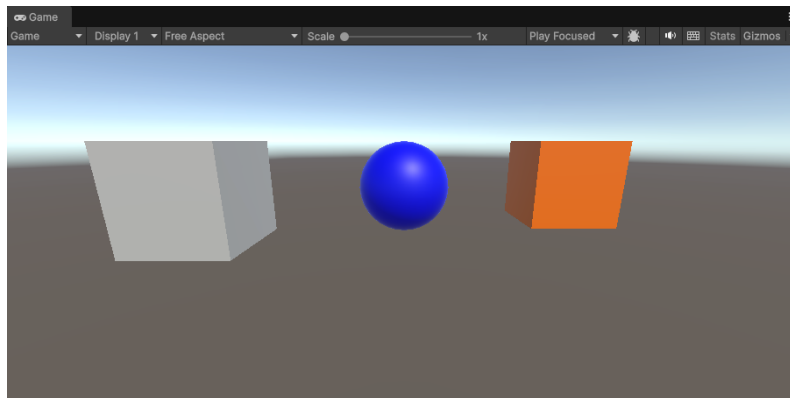
Crea en la carpeta Materials los siguientes materiales, basados en el shader por defecto de URP, Universal Render Pipeline/Lit:

- Azul que utilice el RGB (0, 0, 255).
- Naranja con (255, 120, 0).
- GrisClaro con (200, 200, 200).

Coloca en la escena los siguientes objetos:

- Cubo GrisClaro en L: (-2, 0.5, 0).
- Esfera Azul en L: (0, 0.5, 0.5).
- Cubo Naranja en L: (2, 0.5, 1).

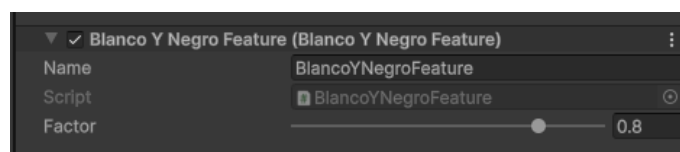
Configura la cámara de la escena en L: (0, 1, -3) y R: (15, 0, 0).



3. Blanco y negro

Implementa una *Render feature* y su pasada correspondiente se ejecute tras el post-procesado y permita convertir la escena a blanco y negro.

La *feature* debe tener un parámetro “factor” entre 0 y 1 que determine el peso que debe tener el color en blanco y negro en el resultado final. Un 0 indica que no tiene peso (se pinta la escena en color) mientras que un factor de 1 significa que la escena se presenta por completo en una escala de grises.



Ten en cuenta que:

- Si el factor configurado es 0, la etapa no deberá ejecutarse.
- La *render feature* no tiene como parámetro el shader/material con que hay que utilizar. Éste será cargado desde código a partir de su nombre (“Hidden/BlancoYNegro”).
- Debe evitarse que el efecto genere más de una pasada en el Frame Debugger; en concreto se debe evitar que tras la conversión del frame completo a blanco y negro se añada una pasada de copia al frame buffer de la cámara.

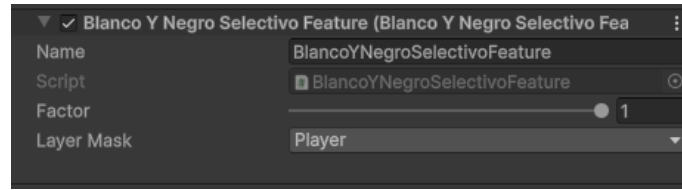
Recuerda que se puede convertir a blanco y negro un color calculando primero su intensidad y utilizando esa intensidad en las tres componentes del color (rgb) para convertirlo a escala de grises. Cada componente no influye por igual en la intensidad final, sino que el verde influye más (0.587), después el rojo (0.299) y por último el azul (0.114).

4. Blanco y negro selectivo

Implementa una nueva *Renderer feature* y su pasada correspondiente que permita convertir a blanco y negro *objetos individuales* en base a la capa a la que pertenezca.

La *feature* tendrá, además del factor a aplicar, la máscara de capas para permitir al usuario seleccionar qué objetos se pintarán en blanco y negro.

Igual que antes:



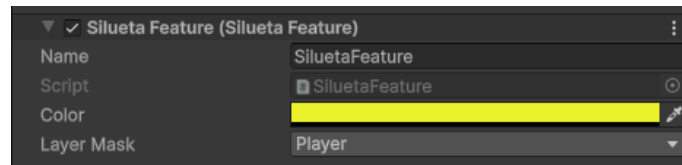
- Si el factor configurado es 0, la etapa no deberá ejecutarse.
- La *render feature* no tiene como parámetro el shader/material con que hay que utilizar. Éste será cargado desde código a partir de su nombre (“Hidden/BlancoYNegroSelectivo”).
- Además, si no hay ningún objeto que pertenezca a las capas seleccionadas, la etapa tampoco se ejecutará.

Crea una nueva *layer* `Player` y haz que la esfera azul de la escena pertenezca a ella de forma que sea el único objeto afectado por el efecto.

5. Silueta

Implementa una nueva *Renderer feature* y su pasada correspondiente que permita convertir pintar la silueta de los objetos que pertenezcan a las capas seleccionadas.

La *feature* permitirá configurar el color de la silueta que se utilizará.



Para implementar el efecto, utiliza la misma idea que utilizamos en la práctica 3. Igual que antes:

- Si no hay ningún objeto que pertenezca a las capas seleccionadas, la etapa tampoco se ejecutará.
- La *render feature* no tiene como parámetro el shader/material con que hay que utilizar. Éste será cargado desde código a partir de su nombre (“Hidden/Silueta”).

6. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- El proyecto de Unity con los directorios mínimos necesarios (`Assets`, `Packages` y `ProjectSettings`). El contenido del directorio `Assets` debe ser el que se deduce de las instrucciones anteriores y que puede verse en la figura 1.
- Un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.